Lab Manual: Insertion Sort, Quick Sort and Chaining

In this lab you will write three C++ programs from scratch. There is no skeleton file. Read the explanation of each topic, then follow the steps of the task. Your output must match the expected output exactly.

Task 1 file name: insertionsortlab.cpp
Task 2 file name: quicksortlab.cpp
Task 3 file name: chaininglab.cpp

Part 1: Insertion Sort

What is insertion sort?

Insertion sort works the way many people arrange playing cards in their hand. You pick up the cards one at a time, and each new card is put in the right place among the cards you are already holding.

In an array, the left part is the sorted part and the right part is not sorted yet. At the start the sorted part is only the first item, because one item alone is always sorted. Every round is called a pass. In each pass:

1. Take the first item of the unsorted part. Keep a copy of it in a variable, call it temp.
2. Look at the sorted items from right to left. Every item that is bigger than temp moves one place to the right. This makes a gap.
3. Stop when you find an item that is not bigger than temp, or when there are no more items on the left. Put temp in the gap.

After every pass the sorted part has one more item. A list of n items needs n - 1 passes. The first pass takes the item at index 1, the last pass takes the item at index n - 1.

Example

Sort the list 5 2 4 1.

```
Start:         5 2 4 1
After pass 1:  2 5 4 1
After pass 2:  2 4 5 1
After pass 3:  1 2 4 5
```

Pass 2 in detail. The sorted part is 2 5. temp is 4, taken from index 2.

```
5 is bigger than 4, so 5 moves one place right:   2 5 5 1   (the gap is at index 1)
2 is not bigger than 4, so stop.
Put 4 in the gap at index 1:                      2 4 5 1
```

Pass 3 in detail. The sorted part is 2 4 5. temp is 1, taken from index 3. The items 5, then 4, then 2 are all bigger than 1 and each moves one place right. There are no more items on the left, so 1 goes to index 0. Notice that you must keep temp in a separate variable, because the first move overwrites the place where 1 was.

Lab Task 1: Insertion Sort (file: insertionsortlab.cpp)

Write these functions.

```
void printList(int list[], int length)
void insertionSort(int list[], int length)
```

- printList: prints the word List: and then all the items, each with one space before it, on one line.
- insertionSort: sorts the list from small to large using insertion sort as explained above. After every pass it prints one line: After pass, the pass number, a colon, and then the whole list in the same style as printList. It prints a line for every pass, even when nothing moved in that pass.

Steps for main

1. Make an int array with these 6 items: 25 12 40 8 19 33.
2. Print the line First list. Call printList, then insertionSort, then printList again.
3. Print an empty line.
4. Make a second int array with these 5 items: 9 7 5 3 1.
5. Print the line Second list. Call printList, then insertionSort, then printList again.

Hint: look at pass 2 of the first list in the expected output. 40 is already in the right place, so the list does not change. Your code must handle this case without moving anything.

Expected output

```
First list
List: 25 12 40 8 19 33
After pass 1: 12 25 40 8 19 33
After pass 2: 12 25 40 8 19 33
After pass 3: 8 12 25 40 19 33
After pass 4: 8 12 19 25 40 33
After pass 5: 8 12 19 25 33 40
List: 8 12 19 25 33 40

Second list
List: 9 7 5 3 1
After pass 1: 7 9 5 3 1
After pass 2: 5 7 9 3 1
After pass 3: 3 5 7 9 1
After pass 4: 1 3 5 7 9
List: 1 3 5 7 9
```

Part 2: Quick Sort

What is quick sort?

Quick sort sorts a list by dividing it into smaller parts. It has two steps that repeat.

Step 1 is called partition. Choose one item of the part as the pivot. Rearrange the part so that all the items smaller than or equal to the pivot are on its left, all the bigger items are on its right, and the pivot itself sits between them. After this the pivot is in its final place and never moves again.

Step 2: the left side and the right side of the pivot are still not sorted. Sort each of them by doing the same thing again. A function that calls itself like this is called recursive. A part with 0 or 1 items is already sorted, so the function stops there.

The partition rule we use

Many ways exist to choose a pivot. In this lab the pivot is always the last item of the part. Partition works on the part from index first to index last:

1. The pivot is list[last]. Make a variable i with the value first - 1. The items at indexes first to i are the ones known to be smaller than or equal to the pivot.
2. Go through the part with an index j from first to last - 1. If list[j] is smaller than or equal to the pivot, add 1 to i and swap list[i] with list[j]. Otherwise do nothing.
3. When the loop ends, swap list[i + 1] with the pivot at list[last]. Now the pivot is at index i + 1.
4. Return i + 1. This is the index of the pivot.

Example

Partition the list 7 2 9 4 6 with first = 0 and last = 4. The pivot is 6 and i starts at -1.

```
j=0  7 is bigger than 6, do nothing                          7 2 9 4 6
j=1  2 is not bigger than 6, i = 0, swap list[0], list[1]    2 7 9 4 6
j=2  9 is bigger than 6, do nothing                          2 7 9 4 6
j=3  4 is not bigger than 6, i = 1, swap list[1], list[3]    2 4 9 7 6
end  swap list[i + 1] = list[2] with the pivot list[4]       2 4 6 7 9
```

The pivot 6 is now at index 2. The left part is indexes 0 to 1 (items 2 4) and the right part is indexes 3 to 4 (items 7 9). Quick sort now does the same on each of these two parts. The left part is sorted first, then the right part.

Lab Task 2: Quick Sort (file: quicksortlab.cpp)

Write these functions.

```
void printList(int list[], int length)
int partition(int list[], int first, int last)
void quickSort(int list[], int length, int first, int last)
```

- printList: prints the word List: and then all the items, each with one space before it, on one line.
- partition: partitions the part from first to last using the rule above and returns the index of the pivot. It does not print anything.
- quickSort: sorts the part from first to last. It only does something when first is less than last. It calls partition, then prints the line Pivot, the pivot value, placed at index, and the pivot index. After that line it calls printList for the whole list (this is why the function gets the length). Then it calls itself for the left part and then for the right part. The length is only used for printing.

Steps for main

1. Make an int array with these 7 items: 38 27 43 3 9 82 10.
2. Call printList.
3. Call quickSort for the whole array (first is 0, last is 6).
4. Call printList again.

Hint: before you run the program, do the first partition by hand, the way the example was done. The pivot is 10. If your first partition does not give 3 9 10 38 27 82 43, your partition function has a mistake.

Expected output

```
List: 38 27 43 3 9 82 10
Pivot 10 placed at index 2
List: 3 9 10 38 27 82 43
Pivot 9 placed at index 1
List: 3 9 10 38 27 82 43
Pivot 43 placed at index 5
List: 3 9 10 38 27 43 82
Pivot 27 placed at index 3
List: 3 9 10 27 38 43 82
List: 3 9 10 27 38 43 82
```

Part 3: Chaining

What is chaining?

In the hashing lab, when two keys got the same slot, the second key was moved to another slot (linear probing). Chaining solves a collision in a different way. The slot never moves the key. Instead, every index of the hash table holds a linked list, and all keys that have the same hash value are stored in the linked list of that index. This linked list is called a chain.

So the hash table is an array of pointers. Each pointer is the first pointer of one chain. If a chain is empty, its pointer is NULL. We use the same node as in the linked list lab:

```
struct nodeType
{
    int info;
    nodeType *link;
};

nodeType *table[7];     // 7 chains, table[i] is the first pointer of chain i
```

The hash function is the same division method, h(key) = key % tableSize.

Inserting a key

1. Calculate h(key). This is the index of the chain.
2. Look through the chain at that index. If the key is already there, it is a duplicate. Do not insert it. Print a message and stop.
3. Otherwise make a new node with the key. Insert it at the front of the chain: the link of the new node gets the old value of table[h], and then table[h] points to the new node. The front is used because it does not need a loop.

Searching for a key

Calculate h(key), then walk through the chain at that index node by node. If a node holds the key, it is found. If you reach NULL, the key is not in the table. Only one chain is searched, not the whole table.

Example

Take a table of size 5 and insert the keys 12, 7, 20, 2, 15, 9 in this order.

```
12 % 5 = 2    chain 2 was empty      chain 2:  12
7 % 5 = 2     goes to the front      chain 2:  7 -> 12
20 % 5 = 0    chain 0 was empty      chain 0:  20
2 % 5 = 2     goes to the front      chain 2:  2 -> 7 -> 12
15 % 5 = 0    goes to the front      chain 0:  15 -> 20
9 % 5 = 4     chain 4 was empty      chain 4:  9
```

The final table:

```
Index 0: 15 -> 20 -> NULL
Index 1: NULL
Index 2: 2 -> 7 -> 12 -> NULL
Index 3: NULL
Index 4: 9 -> NULL
```

Search for 7: 7 % 5 = 2. Look in chain 2. The first node is 2, the second node is 7, found. Search for 17: 17 % 5 = 2. Chain 2 has 2, 7, 12 and then NULL, so 17 is not in the table.

Lab Task 3: Chaining (file: chaininglab.cpp)

Use the nodeType given above. Write these functions.

```
int hashFunction(int key, int tableSize)
int searchChaining(nodeType *table[], int tableSize, int key)
void insertChaining(nodeType *table[], int tableSize, int key)
void printTable(nodeType *table[], int tableSize)
void freeTable(nodeType *table[], int tableSize)
```

- hashFunction: returns key % tableSize.
- searchChaining: looks for the key in its chain. It returns the index of the chain if the key is found, or -1 if it is not. It prints nothing.
- insertChaining: inserts the key at the front of its chain as explained above. After inserting it prints: Inserted, the key, at index, the index. If the key is already in the table, it prints the key followed by is already in the table, and inserts nothing. Use searchChaining to check for the duplicate.
- printTable: prints one line for every index: Index, the index number, a colon, then every key of the chain followed by an arrow (->), and at the end the word NULL. An empty chain prints only NULL.
- freeTable: deletes every node of every chain and sets every table pointer back to NULL. Call it at the end of main so that no memory is left behind.

Steps for main

1. Declare a table of 7 pointers to nodeType and set every pointer to NULL.
2. Insert these keys one by one in this order, using a loop over an array of keys: 25 14 9 32 18 7 21 11 3 13.
3. Try to insert 18 again. It must be reported as a duplicate.
4. Call printTable.
5. Search for 32. If searchChaining returns an index, print the key followed by found at index and the index. Otherwise print the key followed by not found in the table.
6. Do the same search and message for 21, then for 20.
7. Call freeTable.

Hint: because keys are inserted at the front, the chain at index 4 reads 11 18 32 25, which is the reverse of the order in which the keys came.

Expected output

```
Inserted 25 at index 4
Inserted 14 at index 0
Inserted 9 at index 2
Inserted 32 at index 4
Inserted 18 at index 4
Inserted 7 at index 0
Inserted 21 at index 0
Inserted 11 at index 4
Inserted 3 at index 3
Inserted 13 at index 6
18 is already in the table
Index 0: 21 -> 7 -> 14 -> NULL
Index 1: NULL
Index 2: 9 -> NULL
Index 3: 3 -> NULL
Index 4: 11 -> 18 -> 32 -> 25 -> NULL
Index 5: NULL
Index 6: 13 -> NULL
32 found at index 4
21 found at index 0
20 not found in the table
```